

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

Trabalho Prático 1: Jogo da Vida

BCC202 - Estruturas de Dados 1

Fábio Tavares Pinto e Hugo Augusto Silva de Faria
Professor: Pedro Henrique Lopes Silva

Ouro Preto
22 de julho de 2024

Sumário

1	Introdução	1
2	Arquivo de Cabeçalho (automato.h)	1
2.1	Código	1
2.2	Estrutura 'Tabuleiro'	2
2.3	Funções Declaradas	2
3	Arquivo de Implementação (automato.c)	2
3.1	Função alocarReticulado	2
3.2	Função desalocarReticulado	3
3.3	Função LeituraReticulado	4
3.4	Função evoluirReticulado	4
3.5	Função imprimeReticulado	6
4	Arquivo de Implementação (tp.c)	6
4.1	Inclusão de Cabeçalhos	6
4.2	Função main	6
4.3	Chamadas das Funções	7
4.4	Fim da Função main	7
5	Impressões gerais	7
6	Análise	7
6.1	Tempo de Execução	8
6.2	Valgrind	8
7	Conclusão	8

1 Introdução

O Jogo da Vida, criado pelo matemático John Conway em 1970, é um exemplo de autômato celular que demonstra como sistemas simples podem gerar comportamentos complexos e imprevisíveis. Este jogo é baseado em uma grade bidimensional onde cada célula pode estar viva ou morta, e o estado de cada célula é atualizado em iterações discretas com base em regras específicas.

Neste trabalho, desenvolvemos um programa para simular o Jogo da Vida usando a linguagem C. O objetivo principal é criar e manipular um tabuleiro representado por uma matriz bidimensional de inteiros, onde cada célula pode assumir os valores 0 (morta) ou 1 (viva). O programa é projetado para realizar as seguintes operações:

- **Alocação e Desalocação de Memória:** Implementamos funções para alocar e liberar memória para o tabuleiro e sua matriz. Essas funções garantem que o espaço de memória seja gerido eficientemente durante a execução do programa.
- **Leitura e Impressão do Tabuleiro:** Desenvolvemos funções para ler o estado inicial do tabuleiro a partir da entrada padrão e para imprimir o estado do tabuleiro na tela, permitindo a visualização das células vivas e mortas em cada iteração.
- **Evolução do Estado do Tabuleiro:** O núcleo do programa é a implementação das regras do Jogo da Vida, que determinam como as células devem mudar de estado com base no número de vizinhos vivos. Implementamos uma função recursiva para evoluir o tabuleiro para a próxima geração e para executar um número específico de gerações. A recursividade simplifica a lógica do código, tornando-o mais intuitivo e alinhado com a natureza iterativa do Jogo da Vida.

Este relatório detalha a estrutura do programa, as funções implementadas e o funcionamento geral do sistema. Através da implementação deste simulador, buscamos entender melhor como sistemas complexos podem emergir a partir de regras simples e explorar a aplicação prática de autômatos celulares em computação. Além disso, o uso da recursividade na evolução do tabuleiro destaca a elegância e a eficiência de técnicas recursivas em problemas de transformação de estado.

2 Arquivo de Cabeçalho (automato.h)

O arquivo de cabeçalho automato.h define a interface para o módulo que implementa o Jogo da Vida. Ele contém as declarações de tipos de dados e funções que são utilizados para manipular o tabuleiro e executar as operações relacionadas ao jogo. A seguir, estão descritos os principais componentes e suas funções:

2.1 Código

```
1  pragma once // Garante que este arquivo de cabecalho sera incluido apenas uma
    vez no processo de compilacao
2
3  typedef struct {
4      int n; // A dimensao do tabuleiro (n x n)
5      int **matriz; // Ponteiro para uma matriz de inteiros que representa o
        tabuleiro
6  } Tabuleiro;
7
8  // Funcao para alocar memoria para o tabuleiro e a matriz
9  Tabuleiro* alocarReticulado(int n);
10
11 // Funcao para desalocar a memoria do tabuleiro
12 void desalocarReticulado(Tabuleiro **tab);
13
14 // Funcao para ler o tabuleiro
15 void LeituraReticulado(Tabuleiro *tab);
16
```

```

17 // Funcao para evoluir o estado do tabuleiro para a geracao N de acordo com as
    regras do jogo da vida
18 Tabuleiro* evoluirReticulado(Tabuleiro *tab, int n,int geracoes);
19
20 // Funcao para imprimir o estado do tabuleiro na tela
21 void imprimeReticulado(Tabuleiro *tab);

```

2.2 Estrutura 'Tabuleiro'

A estrutura Tabuleiro é definida para representar o estado do tabuleiro do Jogo da Vida. Ela contém:

- 'int n': Um inteiro que representa a dimensão do tabuleiro, sendo 'n x n'.
- 'int **matriz': Um ponteiro para um ponteiro de inteiros, que representa a matriz bidimensional do tabuleiro. Cada célula da matriz pode ser 0 (morta) ou 1 (viva).

2.3 Funções Declaradas

- Tabuleiro* alocarReticulado(int n): Esta função aloca memória para um novo tabuleiro com a dimensão n x n. Ela retorna um ponteiro para a estrutura Tabuleiro recém-alocada. Se a alocação falhar, retorna NULL.
- void desalocarReticulado(Tabuleiro **tab): Esta função libera a memória alocada para um tabuleiro. Ela percorre cada linha da matriz, liberando a memória associada, depois libera a matriz e, finalmente, libera a estrutura Tabuleiro, garantindo que todos os recursos de memória sejam corretamente desalocados.
- void LeituraReticulado(Tabuleiro *tab): Lê o estado inicial do tabuleiro a partir da entrada padrão. Preenche a matriz do tabuleiro com os valores fornecidos, onde cada célula pode ser 0 ou 1.
- Tabuleiro* evoluirReticulado(Tabuleiro *tab, int n,int geracoes): Esta função recursiva atualiza o estado do tabuleiro para a próxima geração com base nas regras do Jogo da Vida de Conway. Ela aloca um novo tabuleiro, aplica as regras de vizinhança para cada célula, desaloca o tabuleiro antigo e chama a si mesma para as gerações restantes, retornando o tabuleiro final após todas as gerações.
- void imprimeReticulado(Tabuleiro *tab): Imprime o estado atual do tabuleiro na tela. Cada célula é exibida com um espaço entre os valores, e cada linha do tabuleiro é impressa em uma nova linha.

3 Arquivo de Implementação (automato.c)

O arquivo de implementação automato.c contém a definição das funções declaradas no arquivo de cabeçalho automato.h. Essas funções são responsáveis pela alocação, desalocação, manipulação e evolução do tabuleiro do Jogo da Vida. A seguir, estão descritas as principais funções e sua implementação:

3.1 Função alocarReticulado

```

1 Tabuleiro *alocarReticulado(int n)
2 {
3     Tabuleiro *tab = (Tabuleiro *)malloc(sizeof(Tabuleiro));
4     if (tab == NULL)
5     {
6         printf("Erro de memoria\n");
7         return NULL;
8     }
9

```

```

10     tab->n = n;
11
12     tab->matriz = (int **)malloc(n * sizeof(int *));
13     if (tab->matriz == NULL)
14     {
15         printf("Erro de memoria\n");
16         free(tab);
17         return NULL;
18     }
19
20     for (int i = 0; i < n; i++)
21     {
22         tab->matriz[i] = (int *)malloc(n * sizeof(int));
23         if (tab->matriz[i] == NULL)
24         {
25             for (int j = 0; j < i; j++)
26             {
27                 free(tab->matriz[j]);
28             }
29             free(tab->matriz);
30             free(tab);
31             printf("Erro de memoria\n");
32             return NULL;
33         }
34     }
35
36     return tab;
37 }

```

A função `alocarReticulado` é responsável por alocar memória para o struct `Tabuleiro` e inicializar sua matriz. Inicialmente, a função aloca memória para o struct `Tabuleiro` e verifica se a alocação foi bem-sucedida. Se houver um erro na alocação, a função imprime uma mensagem de erro e retorna `NULL`. Em seguida, a dimensão do tabuleiro (`n`) é inicializada. A função então aloca memória para a matriz, que é uma matriz 2D de inteiros. Para cada linha da matriz, a função tenta alocar memória e, se houver falha em qualquer ponto, libera toda a memória já alocada e retorna `NULL`, imprimindo uma mensagem de erro. Finalmente, a função retorna um ponteiro para o struct `Tabuleiro` alocado.

3.2 Função `desalocarReticulado`

```

1 void desalocarReticulado(Tabuleiro **tab)
2 {
3     if (tab == NULL || *tab == NULL || (*tab)->matriz == NULL)
4     {
5         return;
6     }
7
8     for (int i = 0; i < (*tab)->n; i++)
9     {
10         free((*tab)->matriz[i]);
11     }
12
13     free((*tab)->matriz);
14     free(*tab);
15     *tab = NULL;
16 }

```

A função `desalocarReticulado` cuida da liberação da memória alocada para o struct `Tabuleiro` e sua matriz. A função começa verificando se o ponteiro do tabuleiro ou a matriz são nulos. Se qualquer um deles for nulo, a função retorna imediatamente. Caso contrário, a função libera a memória alocada para cada linha da matriz. Após liberar todas as linhas, a função libera a memória da própria matriz e do struct `Tabuleiro`. Para evitar acessos posteriores a um ponteiro inválido, a função define o ponteiro do

tabuleiro como NULL. Isso garante que qualquer tentativa de acesso ao tabuleiro após sua desalocação resulte em um ponteiro nulo, ajudando a prevenir erros de memória.

3.3 Função LeituraReticulado

```
1 void LeituraReticulado(Tabuleiro *tab)
2 {
3     for (int i = 0; i < tab->n; i++)
4     {
5         for (int j = 0; j < tab->n; j++)
6         {
7             scanf("%d", &tab->matriz[i][j]);
8         }
9     }
10 }
```

A função `LeituraReticulado` é responsável por ler o estado inicial do tabuleiro a partir da entrada padrão. Ela percorre todas as células da matriz do tabuleiro e usa a função `scanf` para ler o valor de cada célula. Essa função é essencial para inicializar o tabuleiro com os valores que representam o estado inicial de cada célula (viva ou morta).

3.4 Função evoluirReticulado

```
1 Tabuleiro *evoluirReticulado(Tabuleiro *tab, int n, int geracoes)
2 {
3     // Caso base: se não há mais gerações para evoluir, retorne o tabuleiro
4     // atual.
5     if (geracoes == 0)
6     {
7         return tab; // Se o número de gerações restantes é 0, retornamos o
8         // tabuleiro atual, que representa o estado após todas as gerações
9         // solicitadas.
10    }
11
12    // Evolui o tabuleiro para a próxima geração.
13    // Cria um novo tabuleiro para armazenar o estado da próxima geração.
14    Tabuleiro *novoTabuleiro = alocarReticulado(n);
15
16    // Verifica se a alocação foi bem-sucedida.
17    if (novoTabuleiro == NULL)
18    {
19        return NULL; // Se a alocação falhar, retorna NULL. Isso indica um
20        // erro na alocação de memória.
21    }
22
23    // Aplica as regras do Jogo da Vida para definir o próximo estado da
24    // célula (i, j).
25    // Percorre cada célula no tabuleiro atual.
26    for (int i = 0; i < n; i++)
27    {
28        for (int j = 0; j < n; j++)
29        {
30            int vizinhosVivos = 0; // Conta o número de vizinhos vivos da
31            // célula (i, j).
32
33            // Verifica todas as células ao redor da célula atual (i, j),
34            // incluindo diagonais.
35            for (int di = -1; di <= 1; di++)
36            {
37                for (int dj = -1; dj <= 1; dj++)
```

```

31         {
32             // Ignora a propria celula (i, j).
33             if (di == 0 && dj == 0)
34                 continue;
35
36             int ni = i + di; // Calcula a linha do vizinho.
37             int nj = j + dj; // Calcula a coluna do vizinho.
38
39             // Verifica se o vizinho esta dentro dos limites do
40             // tabuleiro.
41             if (ni >= 0 && ni < n && nj >= 0 && nj < n)
42             {
43                 // Adiciona o valor da celula vizinha ao contador de
44                 // vizinhos vivos.
45                 vizinhosVivos += tab->matriz[ni][nj];
46             }
47         }
48
49         // Aplica as regras do Jogo da Vida para atualizar o estado da
50         // celula (i, j).
51         if (tab->matriz[i][j] == 1) // Se a celula esta viva.
52         {
53             // A celula continua viva se tiver 2 ou 3 vizinhos vivos; caso
54             // contrario, morre.
55             if (vizinhosVivos == 2 || vizinhosVivos == 3)
56                 novoTabuleiro->matriz[i][j] = 1; // Continua viva.
57             else
58                 novoTabuleiro->matriz[i][j] = 0; // Morre.
59         }
60         else // Se a celula esta morta.
61         {
62             // A celula se torna viva se tiver exatamente 3 vizinhos vivos
63             // ; caso contrario, permanece morta.
64             if (vizinhosVivos == 3)
65                 novoTabuleiro->matriz[i][j] = 1; // Torna-se viva.
66             else
67                 novoTabuleiro->matriz[i][j] = 0; // Permanece morta.
68         }
69     }
70 }
71
72 // Desaloca o tabuleiro antigo para liberar memoria.
73 desalocarReticulado(&tab);
74
75 // Chama a funcao recursivamente para o proximo numero de geracoes.
76 // Atualiza o tabuleiro para a nova geracao e decrementa o numero de
77 // geracoes restantes.
78 return evoluirReticulado(novoTabuleiro, n, geracoes - 1);
79 }

```

A função `evoluirReticulado` é responsável por evoluir um tabuleiro do Jogo da Vida de Conway através de múltiplas gerações. O propósito desta função é calcular o estado do tabuleiro após um número específico de gerações, utilizando recursão para alcançar o resultado desejado.

A função recebe três parâmetros: um ponteiro para um `Tabuleiro` (`tab`), o tamanho do tabuleiro (`n`), e o número de gerações a serem aplicadas (`geracoes`). O processo inicia verificando se o número de gerações (`geracoes`) é igual a zero. Se for, a função retorna o tabuleiro atual sem alterações, pois não há mais gerações a serem aplicadas.

Se ainda houver gerações a serem processadas, a função continua a execução criando um novo tabuleiro (`novoTabuleiro`) que armazenará o estado da próxima geração. A função `alocarReticulado` é utilizada para alocar memória para este novo tabuleiro. Caso a alocação falhe e `novoTabuleiro` seja

NULL, a função retorna NULL para indicar um erro de alocação de memória.

Com o novo tabuleiro alocado com sucesso, a função procede para aplicar as regras do Jogo da Vida de Conway. Para isso, a função itera sobre cada célula do tabuleiro atual. Para cada célula na posição (i, j), a função conta o número de vizinhos vivos. Esta contagem é feita percorrendo todas as células ao redor da célula atual, incluindo as diagonais, e somando o valor das células vizinhas.

Após calcular o número de vizinhos vivos, a função aplica as regras do Jogo da Vida para determinar o próximo estado da célula (i, j) no novo tabuleiro. Se a célula está viva e possui 2 ou 3 vizinhos vivos, ela permanece viva; caso contrário, ela morre. Se a célula está morta e possui exatamente 3 vizinhos vivos, ela se torna viva; caso contrário, permanece morta.

Com o novo tabuleiro atualizado, a função desaloca o tabuleiro antigo utilizando a função `desalocarReticulado` para liberar a memória previamente alocada. Em seguida, a função chama a si mesma recursivamente para processar a próxima geração, passando o novoTabuleiro como o tabuleiro atual e decrementando o número de gerações restantes.

Finalmente, a função retorna o resultado da chamada recursiva, que é o tabuleiro após o número total de gerações especificado. Este retorno representa o estado final do tabuleiro após todas as gerações.

3.5 Função `imprimeReticulado`

```
1 void imprimeReticulado(Tabuleiro *tab)
2 {
3     printf("\n");
4
5     for (int i = 0; i < tab->n; i++)
6     {
7         for (int j = 0; j < tab->n; j++)
8         {
9             printf("%d ", tab->matriz[i][j]);
10        }
11        printf("\n");
12    }
13 }
```

A função `imprimeReticulado` imprime o estado atual do tabuleiro na tela. Ela percorre todas as células do tabuleiro e usa a função `printf` para imprimir o valor de cada célula, seguido por um espaço. Após imprimir todas as células de uma linha, a função imprime uma nova linha. Isso é repetido para todas as linhas do tabuleiro, resultando em uma representação visual do estado atual do tabuleiro no terminal.

4 Arquivo de Implementação (tp.c)

O arquivo de implementação `tp.c` contém o código principal que utiliza as funções declaradas no arquivo de cabeçalho `automato.h` e implementadas em `automato.c`. Este arquivo orquestra a execução do programa, gerencia a entrada e saída de dados, e controla a evolução do tabuleiro ao longo das gerações. A seguir, está descrito o funcionamento das principais seções e funções deste arquivo:

4.1 Inclusão de Cabeçalhos

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "automato.h"
```

Inclui as bibliotecas necessárias para a entrada e saída padrão e alocação de memória, além do arquivo de cabeçalho `automato.h` que contém as declarações das funções e definições do Tabuleiro.

4.2 Função `main`


```

1 int main()
2 {
3     int n, geracao;
4
5     scanf("%d %d", &n, &geracoes);

```

Declaração das variáveis `n` (dimensão do tabuleiro) e `geracao` (número de gerações a serem evoluídas). Utiliza `scanf` para ler a dimensão do tabuleiro e o número de gerações a partir da entrada padrão.

4.3 Chamadas das Funções

```

1 Tabuleiro *tab = alocarReticulado(n);
2
3 LeituraReticulado(tab);
4
5 tab = evoluirReticulado(tab, n, geracoes);
6
7 imprimeReticulado(tab);
8
9 desalocarReticulado(&tab);

```

- Chama a função `alocarReticulado` para alocar memória para o tabuleiro com a dimensão especificada.
- Chama a função `LeituraReticulado` para ler o estado inicial do tabuleiro a partir da entrada padrão.
- Chama a função `evoluirReticulado` para evoluir o tabuleiro pelo número de gerações especificado. O tabuleiro evoluído substitui o original.
- Chama a função `imprimeReticulado` para imprimir o estado final do tabuleiro.
- Chama a função `desalocarReticulado` para liberar a memória alocada para o tabuleiro.

4.4 Fim da Função `main`

```

1 return 0;
2 }

```

Retorna 0, indicando que o programa terminou com sucesso.

5 Impressões gerais

Durante a implementação deste trabalho, gostei particularmente do desafio de gerenciar a alocação e desalocação de memória, o que me permitiu aprofundar meu conhecimento sobre a manipulação de matrizes dinâmicas em C.

Foi motivador ver o tabuleiro evoluir conforme as regras do Jogo da Vida, destacando a beleza de como regras simples podem resultar em comportamentos complexos. No entanto, encontrei dificuldades na depuração de erros de memória, o que foi inicialmente frustrante, mas extremamente educativo.

Além disso, implementar a recursividade na função de evolução do tabuleiro foi um desafio interessante. A abordagem recursiva não só simplificou a lógica do código, mas também aprimorou meu entendimento sobre como a recursividade pode ser aplicada em algoritmos de transformação de estado.

Este projeto me proporcionou um entendimento mais sólido de autômatos celulares e melhorou minhas habilidades em C, especialmente no que diz respeito ao gerenciamento eficiente de recursos e à aplicação de técnicas recursivas.

6 Análise

Vamos analisar o tempo de execução do código e se teve algum vazamento de memória.

6.1 Tempo de Execução

Testamos o tempo de execução da entrada do teste 20.in. O resultado obtido foi:

```
1 real    0m4.859s
2 user    0m0.003s
3 sys     0m0.001s
```

6.2 Valgrind

Utilizamos o Valgrind para verificar se teve algum vazamento de memória, também com a entrada do teste 20.in. O resultado obtido foi:

```
1 ==159134== HEAP SUMMARY:
2 ==159134==      in use at exit: 0 bytes in 0 blocks
3 ==159134==    total heap usage: 46 allocs, 46 frees, 5,600 bytes allocated
4 ==159134==
5 ==159134== All heap blocks were freed -- no leaks are possible
6 ==159134==
7 ==159134== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

7 Conclusão

Após a implementação e execução do nosso programa, podemos concluir que o desempenho e a eficiência do código foram satisfatórios. Utilizamos o comando time para medir o tempo de execução do nosso programa e observamos que o tempo de execução foi excelente, evidenciando a eficiência das funções implementadas.

Além disso, utilizamos a ferramenta valgrind para verificar possíveis vazamentos de memória durante a execução do programa. Os resultados mostraram que não houve vazamentos de memória, indicando que a alocação e desalocação de memória foram realizadas corretamente em todas as funções do código.

O programa cumpriu com sucesso todos os requisitos propostos, evoluindo corretamente o estado do tabuleiro conforme as regras do Jogo da Vida de Conway. A impressão do tabuleiro nas diversas gerações também foi realizada conforme esperado, proporcionando uma visualização clara da evolução das células ao longo do tempo. A implementação da recursividade na função de evolução do tabuleiro também se mostrou eficaz, simplificando a lógica do código e demonstrando a aplicabilidade de técnicas recursivas em algoritmos de transformação de estado.

Em resumo, o projeto alcançou seus objetivos com um código eficiente e sem problemas de memória, demonstrando um bom entendimento e aplicação das técnicas de alocação dinâmica de memória, manipulação de matrizes e recursividade em C. O resultado final foi obtido com sucesso, validando a eficácia das soluções implementadas.